

Project Blueprint: OmniTalk Edge

Target Track: Arm AI Optimization Challenge (Mobile AI Track)

Core Value Proposition: A 100% offline, multi-modal AI agent that breaks language barriers in zero-connectivity environments through highly optimized on-device inference.

1. Feature Breakdown & User Examples

Feature 1: Live Intercom (Speech-to-Speech Translation)

- **Description:** A low-latency, push-to-talk interface for continuous bidirectional translation. The user speaks in their native language, and the app instantly outputs translated audio.
- **Example Scenario:** A traveler in a rural market needs to negotiate the price of a local craft. The user presses the button, says *"Can you do 500 for this?"* and the phone immediately speaks the local dialect. The seller replies, and the app instantly speaks the English translation back to the user.

Feature 2: Visual Context Scanner

- **Description:** An AR-style camera overlay that detects foreign text, identifies the language, and translates it visually on the screen without taking a photo or connecting to the cloud.
- **Example Scenario:** A user at a transit terminal points their camera at an electronic departures board. The app runs on-device Optical Character Recognition (OCR), translating the foreign text into English so the user can verify their platform number.

Feature 3: Autonomous Agent Mode (The Crown Jewel)

- **Description:** Instead of translating word-for-word, the app acts as an autonomous proxy. The user provides a complex multi-step objective. The app initiates a conversation with a foreign speaker, logically follows up on missing information using a Finite State Machine (FSM), and concludes the interaction, finally providing a written summary to the user.
- **Detailed Example Scenario:**
 - **User Intent:** *"Find out what time the bus to Cox's Bazar leaves, if it has AC, and the ticket price."*
 - **Turn 1 (Agent):** The app speaks Bengali to the clerk: *"Hello, what times do you have buses to Cox's Bazar?"*
 - **Turn 1 (Clerk):** *"We have buses at 8:00 AM, 12:00 PM, and 10:00 PM."*
 - **Turn 2 (Agent):** The FSM checks off "Schedule." It detects that "AC" and "Price" are still unknown. It generates the next query and speaks: *"Do any of those have AC, and how much is a ticket?"*
 - **Turn 2 (Clerk):** *"Only the 10:00 PM has AC. It costs 1500 Taka."*
 - **Turn 3 (Agent):** All criteria met. It speaks: *"Perfect, thank you!"*
 - **Output:** The app displays an English summary for the user: **"Summary: 3 buses available. The 10 PM bus has AC and costs 1500 Taka."**

2. Technology Stack & Model Selection

To satisfy the Arm optimization requirements, the project avoids heavy, generalized models in favor of highly specialized, quantized models compiled with edge-native frameworks.

Pipeline Component	Selected Technology	Architecture / Optimization Rationale
Brain / Reasoning	Llama 3.2 1B (SpinQuant INT4)	Deployed via PyTorch ExecuTorch . Chosen because ExecuTorch natively integrates Arm KleidiAI and XNNPACK, allowing the model to utilize Arm's I8MM (Int8 Matrix Multiplication) and SDOT CPU instructions for rapid prefill speeds.
Listening (ASR)	Whisper.cpp (Tiny - INT8)	Compiles with XNNPACK to leverage NEON instructions, keeping the disk footprint under 75MB.
Speaking (TTS)	Piper TTS (ONNX)	Highly optimized for offline mobile execution (~30MB), producing natural-sounding audio with minimal compute overhead.
Vision (OCR)	PaddleOCR-Lightweight	Integrated with Arm KleidiCV for accelerated frame processing directly on the CPU without bottlenecking the mobile GPU.

3. Feasibility & RAM Management Strategy

Modern Android OS Low Memory Killers (LMK) will forcefully crash background apps if the active memory exceeds the system's threshold. OmniTalk Edge relies on a **Dynamic C++ Memory Swapper** to ensure system stability.

Instead of keeping all models in RAM simultaneously, models are hot-swapped based on the active feature state. According to benchmark data for ExecuTorch running Llama 3.2 1B (SpinQuant):

- The exported `.pte` model size on disk is ~1.08 GB.
- The active memory (RSS) required for the LLM is ~1.92 GB.

Memory Allocation by State:

- **Visual Mode (Active RAM: ~2.2 GB):** PaddleOCR (~150MB) + Llama 3.2 (1.92GB) + App UI/Camera Buffer (~150MB). Whisper and Piper are flushed from memory.
- **Agent Mode (Active RAM: ~2.1 GB):** Whisper ASR (~100MB) + Piper TTS (~50MB) + Llama 3.2 (1.92GB). OCR is flushed from memory.

Given that most mid-tier and flagship mobile devices today ship with 6GB to 8GB of RAM, maintaining a peak footprint of ~2.2 GB is entirely feasible and safe from OS termination.

4. Execution & Latency Estimates

Latency is the defining factor of success. If the Agent Mode takes 10 seconds to reply, the human clerk will walk away. Arm's recent ExecuTorch and KleidiAI integrations provide the following hardware-backed latency guarantees:

Llama 3.2 1B (SpinQuant INT4) on Arm CPU:

- **Time-to-First-Token:** ~0.3 seconds.
- **Prefill Speed (Reading the conversation):** ~260 tokens per second.
- **Decode Speed (Generating the response):** ~50 tokens per second.

Agent Mode Turn Latency Breakdown:

1. **ASR Transcription (Whisper):** ~0.4s
 2. **State Machine Prefill (Llama reading transcript):** ~0.2s (Accelerated by Arm I8MM)
 3. **SLM Decision & Text Generation (30-40 tokens):** ~0.6s
 4. **TTS Generation (Piper TTS):** ~0.3s
- **Total System Reaction Time: ~1.5 Seconds.**

A 1.5-second reaction time is well within natural human conversational pauses, proving to the hackathon judges that cloud-level agentic reasoning is feasible on edge hardware.